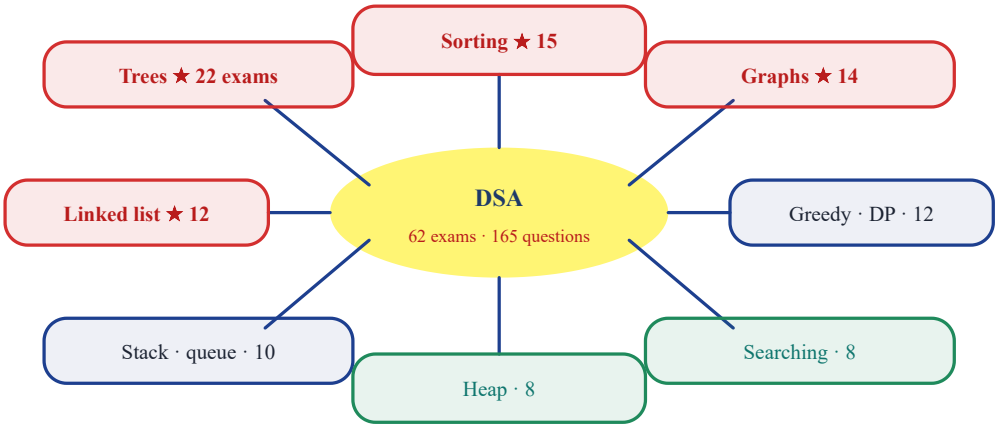
Data Structures & Algorithms

every DSA question asked in BCS · NTRCA · BPSC · bank · BUET-set papers, solved in 23 sheets



↑ the number is how many different exams asked that topic (red = most asked)

How to read a question card

> **Q:** The question, as printed in the paper (typos fixed).
>
> ✎ **Asked in:** the exams it was taken from, e.g. BPSC AP 2023 · 45th BCS CSE 2024

No dated exam → the card names its source (NTRCA question bank, bank IT viva). Green = worked result / output.

Most asked

① array vs linked list (5)
② traversal → rebuild tree
③ BFS vs DFS
④ quick / merge sort cost
⑤ stack vs queue
⑥ D&C vs DP
⑦ build a heap

Syllabus coverage

"Internal data representation; Abstract data types; Elementary data structures: **arrays, lists, stacks, queues, trees, graphs**; Advanced: **heaps**, Fibonacci heaps, B-trees; Recursion, **sorting, searching, hashing**; analysis of algorithms; design: **divide and conquer, greedy, dynamic programming**; graph algorithms; NP-completeness".

> **Legend:** **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked answer / output

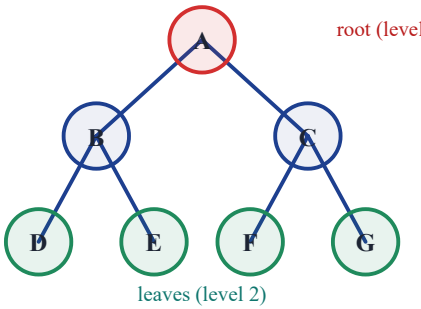
Contents

✓ Trees I–IV …… p. 2–5
✓ Sorting I–III …… p. 6–8
✓ Graphs I–III …… p. 9–11
✓ Linked list …… p. 11–12
✓ Stack and queue …… p. 13–14
✓ Heap; searching, hashing … p. 14–16
✓ Algorithm design I–III …… p. 17–19
✓ Notation, basics, arrays … p. 20–21
✓ Night before …… p. 22
✓ Practice …… p. 23

1 Binary trees and how many nodes

Q1: What is a binary tree? Explain with an example, and discuss the methods of traversing it.

✎ **Asked in:** WZPGCL AE(CSE) 2023 · 18th NTRCA College ICT 2024



A **binary tree** is a tree in which each node has **at most two children**, called left and right. Example: an expression tree, a BST, a heap.

Traversal	Rule	On the tree
Preorder	Root, L, R	A B D E C F G
Inorder	L, Root, R	D B E A F C G
Postorder	L, R, Root	D E B F G C A
Level order	level by level (queue)	A B C D E F G

Q2: Maximum and minimum number of nodes of a binary tree of height 7, the root at height 0.

✎ **Asked in:** NESCO AM(ICT) 2021

Max: every level full → $2^0 + \dots + 2^7 = 2^8 - 1$. Min: one node per level → h + 1.

→ max **255**, min **8**

Q3: Maximum number of nodes in a binary tree of N levels? Minimum number of nodes in a binary tree?

✎ **Asked in:** Combined Bank AP 2020 · BCC AP 2023

Level i holds at most 2^{i-1} nodes (levels from 1).

→ **$2^N - 1$** · minimum **0** (empty tree is a binary tree); with height h: h + 1

Q4: A proper binary tree has one more leaf than internal nodes: prove it.

✎ **Asked in:** Titas Gas AE(CSE) 2024 (printed "one more node is internal node")

In a **proper (full) binary tree** every internal node has exactly 2 children. Let i = internal nodes, l = leaves, n = i + l. Every node except the root has one parent edge: edges = n − 1. Every internal node gives 2 edges: edges = 2i. So 2i = i + l − 1 → **l = i + 1**. Check on the tree above: i = 3 (A, B, C), l = 4 ✓.

Binary tree type	Rule	Note
Full (proper)	0 or 2 children each	leaves = internal + 1
Complete	all full but the last, filled left	heaps, arrays
Perfect	every level completely full	$2^{h+1} - 1$ nodes
Skewed	one child per node	like a list, height n − 1

Q5: How do you represent a binary tree using an array?

✎ **Asked in:** BGDCL AM(CSE) 2024 · MRA SCO 2023

index	1	2	3	4	5	6	7
node	A	B	C	D	E	F	G

Memory trick: Node at i: **left 2i**, **right 2i + 1**, parent ⌊i/2⌋ (root at 1). Root at 0: 2i + 1, 2i + 2.

Store level by level; a missing child leaves an **empty slot**, so a skewed tree of height h wastes up to $2^{h+1} - 1 - (h + 1)$ cells: arrays suit **complete** trees (heaps); otherwise use linked nodes (data, left, right).

2 Rebuild a tree from its traversals

Q6: In-order and preorder traversals of a binary tree are dbeafcg and abdecfg. What is the post-order traversal?

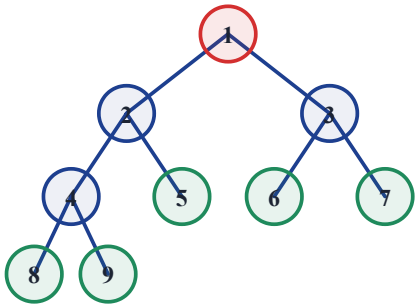
✎ **Asked in:** 45th BCS CSE 2024

1. Preorder's first letter is the root: **a**. In inorder, left of a = d b e, right = f c g
2. Next in preorder: **b** roots the left part (d | b | e); then **c** roots the right part (f | c | g)
3. Tree: a(b(d, e), c(f, g)); read Left-Right-Root

→ post-order **d e b f g c a**

Q7: pre[] = {1, 2, 4, 8, 9, 5, 3, 6, 7}, post[] = {8, 9, 4, 5, 2, 6, 7, 3, 1}: draw the binary tree.

✎ **Asked in:** BPDB AE(CSE) 2024 · BUET M.Sc. April 2016



1. Root = pre[0] = 1 = last of post
2. pre[1] = 2 is the left child; in post, everything up to 2 (8 9 4 5 2) is the left subtree, the rest (6 7 3) the right
3. Repeat: 2 → (4 → 8, 9), 5; 3 → 6, 7
4. Check inorder: 8 4 9 2 5 1 6 3 7 ✓

Exam trap: Pre + post fix the tree **only when it is full** (every node 0 or 2 children); with a one-child node, left and right cannot be told apart. Pre + in, or post + in, always work. Say this in one line.

Q8: Construct the tree from preorder 1 5 8 7 4 9 3 and inorder 8 5 7 1 9 4 3.

✎ **Asked in:** APSCL AE(ICT) 2021 (printed "construct a BST")

Root 1; left (in 8 5 7, pre 5 8 7) → 5(8, 7); right (in 9 4 3, pre 4 9 3) → 4(9, 3). Tree **1(5(8, 7), 4(9, 3))**, postorder 8 7 5 9 3 4 1. It is a binary tree, **not a BST** (8 > 5 sits on the left of 5): state it.

Q9: A binary tree with nodes a … i has post-order a b f c h d e g i. Draw it and give its sequential (array) representation.

✎ **Asked in:** Petrobangla AM(CSE) 2024 (three sources)

Not unique in general; an array form implies a **complete** tree of 9 nodes. Its postorder positions are 8 9 4 5 2 6 7 3 1, so fill them with a b f c h d e g i: root i; i → (h, g); h → (f, c); g → (d, e); f → (a, b). **Right, left, root** ✎ **Asked in:** DPDC AM(ICT) 2025 (tree not printed): the preorder read backwards; on the tree of p. 2 → G F C E D B A. · **Traversals of a 15-node tree** ✎ **Asked in:** BPSC AP 2023: draw a full tree of height 3 (1 … 15 level by level) and apply the table of p. 2.

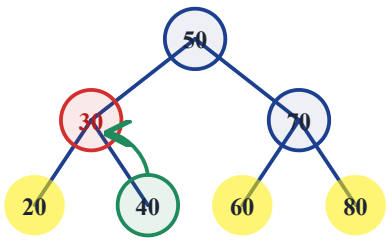
1	2	3	4	5	6	7	8	9
i	h	g	f	c	d	e	a	b

BFS and DFS of the tree 1 → (2, 3), 2 → (4, 5) ✎ **Asked in:** BAPEX AGM(ICT) 2023: BFS 1 2 3 4 5 · DFS (preorder) 1 2 4 5 3 · inorder 4 2 5 1 3 · postorder 4 5 2 3 1. · Pre + in steps ✎ **Asked in:** BSCL AE(CSE) 2026: Q6's method.

3 Binary search tree operations

Q10: Explain binary search tree operations. Insert data into a BST, print it in post-order, delete one node and redraw the valid BST.

✎ **Asked in:** 17th NTRCA CSE 2023 · Combined Bank SO(IT) 2020 · insert, delete, redraw: PGCB AE(CSE) 2024 (data not printed)



delete 30 → successor 40 moves up

- ✓ **Search**: compare with root, go left if smaller, right if larger; O(h)
- ✓ **Insert** 50, 30, 70, 20, 40, 60, 80: search, attach where it falls off
- ✓ Post-order: 20 40 30 60 80 70 50
- ✓ **Delete**: leaf → remove; one child → link child up; two children → replace by **inorder successor** (smallest of right subtree)
- ✓ Delete 30 → 50(40(20), 70(60, 80)); post-order 20 40 60 80 70 50

Q11: Write an algorithm to find a node in a binary search tree.

✎ **Asked in:** PSB AP 2018

```
Node* search(Node *t, int key) {
    while (t != NULL && t->key != key)
        t = (key < t->key) ? t->left : t->right;
    return t;              /* NULL = not found; O(h): O(log n) balanced, O(n) skewed */
}
```

Memory trick: "**Inorder of a BST is sorted**": 20 30 40 50 60 70 80. That is why the MCQ "which traversal sorts a BST?" is **inorder** ✎ **Asked in:** 49th BCS ICT 2025.

4 B-tree and B+ tree

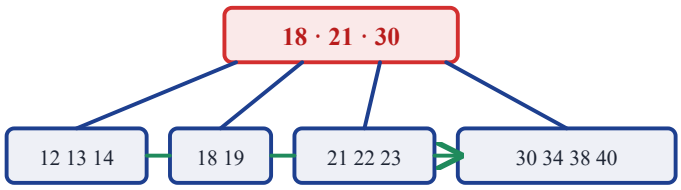
Q12: What is a B-tree data structure? Explain its uses.

✎ **Asked in:** 17th NTRCA CSE 2023 · B+ tree in indexing: NTRCA question bank

A **B-tree of order m** is a balanced multi-way search tree: each node holds up to m − 1 sorted keys and m children, every non-root node is at least half full, all leaves are on the same level, height O(log n). Few, wide nodes = few **disk reads**: used for **database and file-system indexes** (MySQL InnoDB, NTFS). A **B+ tree** keeps all keys in linked leaves and only copies in internal nodes, so range scans are fast.

Q13: Draw a B+ tree for the keys 12, 14, 19, 30, 21, 22, 13, 34, 18, 23, 38, 40 with n = 5.

✎ **Asked in:** BUET M.Sc. April 2017



n = 5 pointers → at most **4 keys** per node. A full leaf splits 3 | 2 and **copies** the right half's first key up: inserting 21 splits [12 14 19 21 30] → root 21; 18 splits [12 13 14 18 19] → 18 up; 23 splits [21 22 23 30 34] → 30 up. 38, 40 fill the last leaf.

Decision tree ✎ **Asked in:** 17th NTRCA CSE 2023: a tree whose internal nodes are tests and leaves are outcomes (e.g. "outlook? → humidity? → play"). Sorting by comparisons is a decision tree with n! leaves, height ≥ $\log_2 n!$ = Ω(n log n): the MCQ "lower bounds in sorting are proved with a **decision tree**" ✎ **Asked in:** 49th BCS ICT 2025.

5 One table answers ten questions

Q14: What is the complexity of an algorithm? Give the best, worst and average time complexities of the common sorting algorithms.

✎ **Asked in:** 17th NTRCA CSE 2023 · DPDC AM(ICT) 2025 · worst case of bubble, quick, merge: BTCL JAM 2022 · merge, heap, insertion: BBL AM(ICT) 2019

Algorithm	Best	Average	Worst	Space	Stable	Note
Bubble	n	n^2	n^2	1	yes	best with the "no swap → stop" flag
Selection	n^2	n^2	n^2	1	no	fewest swaps (n − 1)
Insertion	**n**	n^2	n^2	1	yes	best for small / nearly sorted
Merge	n log n	n log n	n log n	n	yes	guaranteed, needs extra array
Quick	n log n	n log n	**n^2**	log n	no	fastest in practice
Heap	n log n	n log n	n log n	1	no	in place and guaranteed
Counting / radix	n + k	n + k	n + k	n + k	yes	not comparison-based

Complexity = how the running time (time complexity) or memory (space complexity) of an algorithm grows with the input size n, written with **Big-O** for the upper bound. Any comparison sort needs **Ω(n log n)** in the worst case.

MCQs asked: Worst-case comparisons of insertion sort → **n(n − 1)/2** ✎ **Asked in:** 49th BCS ICT 2025 · not comparison-based → **counting sort** ✎ **Asked in:** 49th BCS ICT 2025 · fastest to sort S = {1, 2, 3, 8, 15, 10} → **insertion sort**: nearly sorted, 6 comparisons ✎ **Asked in:** Combined Bank AP 2020

Exam trap: "Fastest sorting algorithm?" ✎ **Asked in:** BCC AP 2023 has no single answer: **quick sort** on average in practice, **merge / heap** for a guaranteed n log n, **insertion** on nearly sorted data, **counting / radix** for small integer keys. Write all four lines.

Q15: Write the bubble sort algorithm and find its time complexity.

✎ **Asked in:** BPSC AP 2021 · why bubble sort is slow for big data vs merge sort: Bank AP written (bank name cut off on the sheet)

```
for (i = 0; i < n-1; i++) {
  swapped = 0;
  for (j = 0; j < n-1-i; j++)
    if (a[j] > a[j+1]) {
      swap(a[j], a[j+1]);
      swapped = 1;
    }
  if (!swapped) break;
}
```

5 8 3 6 2 →
pass 1: 5 3 6 2 **8**
pass 2: 3 5 2 **6 8**
pass 3: 3 2 **5 6 8**
pass 4: **2 3 5 6 8** (10 comparisons)

Comparisons (n − 1) + (n − 2) + … + 1 = **n(n − 1)/2** → **O(n²)**; already sorted → one pass, O(n). For 10^6 items $n^2 = 10^{12}$ steps vs n log n ≈ 2×10^7 for merge sort.

The printed question "why does bubble sort fail if the array is unsorted?" has a wrong premise: bubble sort works on any array. **Binary search** is the one that fails on unsorted data.

6 Quick sort

Q16: Write the recursive pseudo-code of quick sort and its average complexity. How is quick sort implemented?

✎ **Asked in:** BPSC ADA 2022 · BPSC AME 2021 · how quick sort works: NTRCA registration viva 2023

```
quickSort(A, lo, hi):
  if lo < hi:
    p = partition(A, lo, hi)
    quickSort(A, lo, p - 1)
    quickSort(A, p + 1, hi)
partition(A, lo, hi):      // Lomuto
  pivot = A[hi]; i = lo - 1
  for j = lo .. hi-1:
    if A[j] < pivot: i++; swap A[i], A[j]
  swap A[i+1], A[hi]; return i + 1
```

10 80 30 90 40 50 70, pivot 70 →
10 30 40 50 **70** 90 80 (70 is final)
sorted: 10 30 40 50 70 80 90

Average: balanced splits, T(n) = 2T(n/2) + n → **O(n log n)**; in place, O(log n) stack.

Q17: Why is the worst-case complexity of quicksort $O(n^2)$? Analyse it when the first element is the pivot and the array is already sorted.

✎ **Asked in:** 45th BCS CSE 2024 · Petrobangla AM(CSE) 2024 · randomized quicksort worst case (MCQ): Combined Bank Officer(IT) 2019

Sorted input + first pivot: the pivot is the smallest, so the split is **0 | n − 1** every time. $T(n) = T(n-1) + (n-1) = (n-1) + (n-2) + \ldots + 1 =$ **$n(n-1)/2 = O(n^2)$** (counted: n = 10 → 45 comparisons). Random pivot makes it unlikely, but the **worst case is still $O(n^2)$**; expected O(n log n).

7 Merge sort

Q18: Write the merge sort algorithm using divide and conquer. Explain how $T(n) = 2T(n/2) + n$ is derived.

✎ **Asked in:** 45th BCS CSE 2024 · recurrence: BPSC AP 2023

```
mergeSort(A, l, r):
  if l < r:
    m = (l + r) / 2
    mergeSort(A, l, m)      // T(n/2)
    mergeSort(A, m+1, r)    // T(n/2)
    merge(A, l, m, r)       // n
```

Divide costs O(1), **two halves** cost 2T(n/2), **merge** touches each element once: n. Hence $T(n) = 2T(n/2) + n$. Recursion tree: $\log_2 n$ levels × n work each = **O(n log n)** (master theorem case 2).

38 27 43 3 9 82 10 → 3 9 10 27 38 43 82

Q19: Write a linear algorithm to merge two sorted lists. Why is it O(n)?

✎ **Asked in:** SGFL AE(IT) 2023

```
i = j = k = 0
while i < n and j < m:
  Z[k++] = (X[i] <= Y[j]) ? X[i++] : Y[j++]
copy the rest of X, then the rest of Y
```

Each step moves one element to Z and never goes back: at most **n + m − 1 comparisons** → O(n + m). X = 2 5 9, Y = 1 6 7 10 → 1 2 5 6 7 9 10 (6 comparisons)

Quick vs merge ✎ **Asked in: BPSC ME 2021**	**Quick sort**	**Merge sort**
Time	n log n average, **n^2** worst	n log n always
Space	in place, O(log n) stack	O(n) extra array
Work done in	partition (before recursion)	merge (after recursion)
Stable / best for	no / arrays in RAM	yes / linked lists, external sorting

Best and worst cases ✎ **Asked in:** RPGCL AM(ICT) 2022: binary search O(1) / O(log n) · quick sort O(n log n) / $O(n^2)$ (the printed O(n) best case is wrong) · DFS O(V + E) both.

Q20: Write the selection sort algorithm and sort 45, 72, 80, 65, 84, 52, 37 in descending order, step by step.

✎ **Asked in:** NTRCA question bank

```
for (i = 0; i < n-1; i++) {
  m = i;                        /* largest so far */
  for (j = i+1; j < n; j++)
    if (a[j] > a[m]) m = j;
  swap(a[i], a[m]);
}
```

Pass	Array after the pass
1	**84** 72 80 65 45 52 37
2	**84 80** 72 65 45 52 37
3, 4	no change (72, 65 already in place)
5	**84 80 72 65 52** 45 37
6	**84 80 72 65 52 45 37**

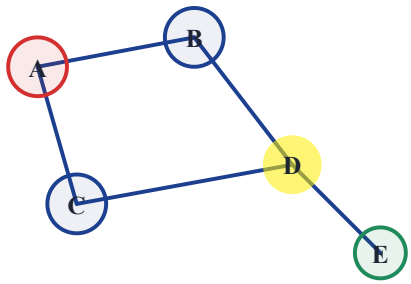
Always $n(n-1)/2 = 21$ **comparisons** for $n = 7$, even on sorted input, so best = worst = $O(n^2)$; at most $n - 1$ swaps, which is why it suits data that is costly to move.

8 BFS vs DFS

Q21: What are the differences between BFS and DFS? For what sort of problems is it better to use BFS than DFS?

✎ **Asked in:** 45th BCS CSE 2024 · BPSC AP 2021 · 17th NTRCA CSE 2023 · BPSC AP 2022

Point	BFS	DFS
Explores	level by level	one branch to the end, then backtracks
Uses	**queue**	**stack** / recursion
Time	O(V + E)	O(V + E)
Memory	a whole level (wide)	one path (deep)
Gives	shortest path (unweighted)	cycles, topological sort, SCC



BFS from A: A B C D E · DFS: A B D C E

Use BFS when the answer is **near the start** or you need the **fewest edges**: shortest path in an unweighted graph, friends within 2 hops, web crawling by depth, peer-to-peer search. Use DFS for deep searches, cycle detection, puzzles, topological order.

Exam trap: A common printed answer says DFS uses a stack "for BFS traversal" and that "BFS is slower": both wrong. Both are O(V + E); **BFS → queue** (MCQ ✎ **Asked in:** 49th BCS ICT 2025), **DFS → stack**.

Q22: Explain BFS traversal of a graph with an example.

✎ **Asked in:** 17th NTRCA ICT 2023 · DBBL PO 2017

Queue [A] → take A, add B, C → take B, add D → take C (D seen) → take D, add E → take E. Order A B C D E; levels A | B C | D | E.

Q23: Why is depth-first search not optimal?

✎ **Asked in:** 45th BCS ICT 2024

DFS returns the **first** path it reaches, not the shortest: it may go deep down A → B → D → E when A → C → D was cheaper, and on infinite / very deep branches it may never return. BFS (unit costs) and uniform-cost / A* are optimal.

Q24: Which problems are solved more efficiently with an adjacency list than an adjacency matrix, and the other way round?

✎ **Asked in:** Combined Bank AP 2023 (printed wording garbled)

Need	Adjacency list	Adjacency matrix
Space	**O(V + E)**: sparse graphs (roads, social)	O(V^2): dense graphs
Is there an edge u–v?	O(degree)	**O(1)**
Visit all neighbours (BFS, DFS, Dijkstra)	**O(degree)**	O(V) per vertex
Floyd–Warshall, dense-graph Prim	slower	**natural fit**

Q25: Give the adjacency list of a complete binary tree on 7 vertices and an equivalent adjacency matrix; vertices numbered 1 … 7 as in a binary heap.

✎ **Asked in:** BB AP 2023

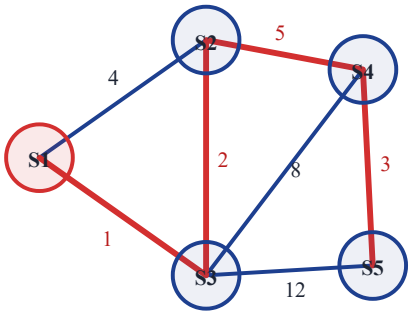
Heap numbering: i is joined to 2i and 2i + 1. List: 1 → 2, 3 · 2 → 1, 4, 5 · 3 → 1, 6, 7 · 4 → 2 · 5 → 2 · 6 → 3 · 7 → 3. The matrix is symmetric (undirected), 12 ones = 2 × 6 edges.

```
   1 2 3 4 5 6 7
1  0 1 1 0 0 0 0
2  1 0 0 1 1 0 0
3  1 0 0 0 0 1 1
4  0 1 0 0 0 0 0
5  0 1 0 0 0 0 0
6  0 0 1 0 0 0 0
7  0 0 1 0 0 0 0
```

9 Shortest path: Dijkstra (greedy)

Q26: Define feasible and optimal solution. Write the greedy algorithm for the single-source shortest path. / S1 … S5 are nodes, line values are power-transmission costs: find the shortest paths and the average cost.

✎ **Asked in:** 45th BCS CSE 2024 · NESCO AM(ICT) 2018 (graph not printed: sample below)



Feasible = satisfies all constraints (any S1 → S5 path); **optimal** = the feasible one with the best value (cheapest).

1. d[S1] = 0, others ∞; pick the unvisited vertex with the **smallest d**
2. relax its edges: d[v] = min(d[v], d[u] + w)
3. S1 0 → S3 1 → S2 3 (via S3) → S4 8 → S5 11 (via S4; via S3 it would be 1 + 12 = 13)

→ S2 = 3 (S1→S3→S2), S3 = 1, S4 = 8 (S1→S3→S2→S4), S5 = 11 (S1→S3→S2→S4→S5) · average cost (3 + 1 + 8 + 11) / 4 = **5.75**. Time O((V + E) log V) with a heap; no negative edges (then Bellman–Ford).

Draw a weighted graph from an adjacency representation ✎ **Asked in:** BUET M.Sc. April 2017 · BUET M.Sc. May 2016 (representation not printed): one circle per vertex, one line per non-zero entry (once if symmetric, an arrow if not), the entry's value written on the line; e.g. the S1 … S5 graph of p. 10 is the list S1: S2/4, S3/1 · S2: S3/2, S4/5 · …

10 Minimum spanning tree: Kruskal

Q27: Find the minimum spanning tree of the graph using Kruskal's algorithm.

✎ **Asked in:** RAKUB Programmer 2021 (graph not printed: sample in the table)

Edges sorted by weight	Take?
BC 1 · BD 2 · DE 2 · AC 3	**take all** (no cycle)
AB 4 · CD 4 · CE 5	skip: would close a cycle
EF 6	**take** → 5 edges for 6 vertices, stop

MST = BC, BD, DE, AC, EF, total **14**. Kruskal: sort edges, add the cheapest that joins two different components (union-find); O(E log E) ✎ **Asked in:** MCQ: coaching MCQ set, no exam tag. Prim grows one tree from a vertex instead.

Suffix tree MCQ ✎ **Asked in:** 49th BCS ICT 2025: a compressed trie of all suffixes of a string → **fast string (substring) search** in O(m).

11 Array vs linked list

Q28: Differentiate between a linked list and an array. Write the advantages and disadvantages of a linked list over an array.

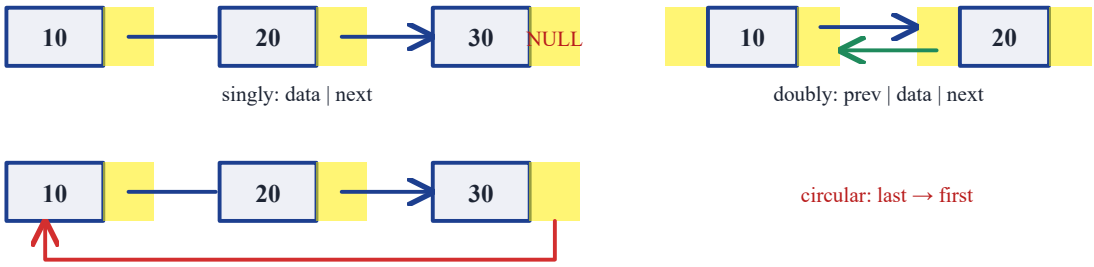
✎ **Asked in:** 17th NTRCA ICT 2023 · BNPS AP 2017 · BPSC AP 2021 · BPSC AP 2023 · BPSC ME 2021 · advantages / disadvantages: 45th BCS CSE 2024 · bank IT viva

Point	Array	Linked list
Memory	**contiguous**, fixed size	scattered nodes, grows / shrinks
Access k-th	**O(1)** by index	O(n), walk from head
Insert / delete (position known)	O(n), shift elements	**O(1)**, change pointers
Extra space	none	a pointer per node
Cache, binary search	cache-friendly; binary search works	poor locality; no fast binary search

Memory trick: "**Array = Address, List = Links**": arrays win at **reading by index**, lists win at **inserting and deleting**. For the NESCO question "insert, delete or search?" ✎ **Asked in:** NESCO AM(ICT) 2021 → linked list is better for **insert and delete**, worse for search.

Q29: What is a linked list? Give the algorithm to create one with an example drawn graphically, and describe its types with diagrams.

✎ **Asked in:** 17th NTRCA CSE 2023 · BPSC AE 2022 · 18th NTRCA College ICT 2024 · singly, doubly, circular: BNPS AP 2017




```
struct Node { int data; struct Node *next; };
head = NULL;                        /* create 10 → 20 → 30 */
for each value x (taken in reverse: 30, 20, 10):
    n = malloc(sizeof *n);
    n->data = x; n->next = head;  /* insert at front */
    head = n;
```

A **linked list** is a linear collection of **nodes**, each holding data and a pointer to the next; the list is reached through **head** and ends in NULL. Doubly lists also point back (easy reverse walk, delete given a node); circular lists link the last node to the first (round-robin queues).

Q30: Can binary search be applied on a sorted linked list? Justify with an example.

✎ **Asked in:** 45th BCS CSE 2024

Possible, but **not efficient**: finding the middle needs walking n/2 nodes, so total O(n), no better than linear search. On 3 → 8 → 15 → 21 → 34 the middle (15) costs 2 hops, the next middle 1 more… A **skip list** or BST gives O(log n).

Q31: The list 3, 5, 7, 11, 13, 17 is stored in sequence. Which of queue, heap, linked list, stack needs the least data movement to remove 11? Justify.

✎ **Asked in:** 38th BCS 2018

Linked list: set 7's next to 13, nothing moves. Stack / queue must pop and push the others; an array shifts 13, 17; a heap re-heapifies.

Q32: Can divide and conquer be applied to linked lists?

✎ **Asked in:** bank IT viva

Yes: **merge sort** is the natural sort for a linked list. Split at the middle with a slow and a fast pointer (O(n)), sort both halves, merge by relinking nodes: O(n log n) and no extra array. Binary search gains nothing, because reaching the middle already costs O(n).

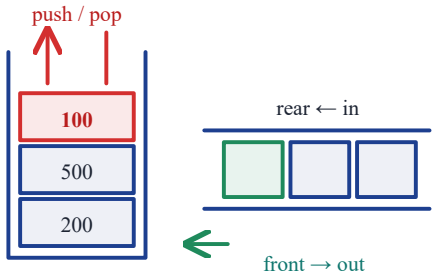
```
slow = head; fast = head->next;
while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
}   /* slow = end of first half */
```

12 Stack vs queue

Q33: Write the differences between stack and queue, and two problems solved by each.

✎ **Asked in:** BPSC Lecturer 2017 · Combined Bank AP 2024 · DESCO SAE 2023 · 18th NTRCA College ICT 2024 · bank IT viva

Point	Stack	Queue
Order	**LIFO**	**FIFO**
Ends used	one (top)	two (rear in, front out)
Operations	push, pop, peek	enqueue, dequeue, front
Problems	function calls / recursion, undo, bracket matching, postfix evaluation	CPU / printer scheduling, BFS, buffers, call-centre waiting line



Q34: Push(200), Push(500), Push(100), S = Pop(). What is S?

✎ **Asked in:** BAPEX AGM(ICT) 2023

→ S = **100** (last in, first out); 500 is now the top

Q35: Describe the stack operations briefly. Why do we use a stack?

✎ **Asked in:** BPSC SAE 2021 · BNPS AP 2017 · push and pop (MCQ): BREB 2019 · PKB Programmer 2019

push: top++ then a[top] = x (overflow if full) · **pop**: x = a[top--] (underflow if empty) · **peek**, **isEmpty**: all O(1). Used whenever the **latest** item must be handled first.

Q36: Write the steps of the enqueue operation of a queue.

✎ **Asked in:** Combined Bank Officer(IT) 2020

1. If the queue is full (rear = MAX − 1; circular: (rear + 1) % MAX = front) → **overflow**, stop
2. If it is empty (front = −1) set front = 0
3. rear = rear + 1 (circular: (rear + 1) % MAX)
4. queue[rear] = item; done in O(1)

Q37: A circular queue has 6 locations (0 – 5), initially empty. Add A, C, D; insert F; delete two; add K, L, M; delete two; add R. Give FRONT and REAR after each step.

✎ **Asked in:** 38th BCS 2018

Step	start	+A C D	+F	−2	+K L M	−2	+R
FRONT	−1	0	0	2	2	4	**4**
REAR	−1	2	3	3	0 (M wraps)	0	**1**

→ final FRONT = 4, REAR = 1; the queue holds L (4), M (0), R (1). The wrap-around is the whole point of a circular queue.

Q38: Explain the logic to implement a FIFO queue using only two stacks.

✎ **Asked in:** Bank AP written (bank name cut off on the sheet)

enqueue(x): push x on **in**. **dequeue()**: if **out** is empty, pop everything from in and push it on out (this reverses the order), then pop out. Each item moves at most twice → **O(1) amortised**. enq 1, 2, 3; deq → 1; enq 4; deq → 2, 3, 4. Reverse question (MCQ): a stack needs **two queues**.

13 Heaps

Q39: Write down the properties of a max heap, and the heapsort algorithm.

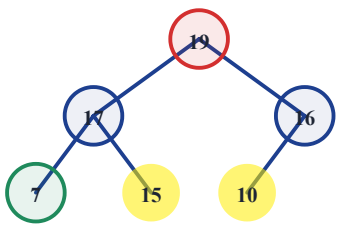
✎ **Asked in:** BPSC SCO 2022

- ✓ **Shape**: a **complete** binary tree (levels full, last filled left to right) → stored in an array
- ✓ **Order**: parent ≥ each child, so the **maximum is at the root**
- ✓ height $\lfloor \log_2 n \rfloor$: insert and delete-max O(log n), find-max O(1)
- ✓ no order between siblings (it is not a BST)

```
heapSort(A, n):
  for i = n/2 - 1 down to 0:     // build: O(n)
      siftDown(A, n, i)
  for end = n-1 down to 1:
      swap(A[0], A[end])         // max to the back
      siftDown(A, end, 0)        // O(log n)
// total O(n log n), in place, not stable
```

Q40: Build a heap from {15, 19, 10, 7, 17, 16} and redraw it after deleting 7.

✎ **Asked in:** PGCB AE(CSE) 2021



max-heap by insertion: 19 17 16 7 15 10

1. Insert one by one, **sift up**: 15 · 19 swaps up · 10 · 7 · 17 passes 15 · 16 passes 10
2. Delete 7: put the last element (10) in its place, sift: 10 < 17 parent, no children → stays
3. Result 19 / 17, 16 / 10, 15

The printed solution draws a BST, not a heap. If a min-heap is meant: 7 / 10, 15 / 19, 17, 16; deleting the root 7 gives 10 / 16, 15 / 19, 17.

Q41: For the items 40, 45, 80, 90, 50, 70 draw the heap and the binary search tree.

✎ **Asked in:** SGFL AE(IT) 2023

Max-heap by insertion: **90 / 80, 70 / 40, 50, 45**
(bottom-up heapify gives 90 / 50, 80 / 45, 40, 70: also valid)
Heap sort output: 40 45 50 70 80 90

BST: 40 → right 45 → right 80 → (50 → right 70, 90)
inorder 40 45 50 70 80 90 = sorted ✓
height 4: an unbalanced, nearly skewed tree

Heap sort vs merge sort ✎ **Asked in: BPSC AP 2021 · BBL AM(ICT) 2019**	**Heap sort**	**Merge sort**	**Insertion sort**
Time (worst)	n log n	n log n	n^2 (n if nearly sorted)
Extra space	**O(1)**	O(n)	O(1)
Stable	no	**yes**	yes
Advantage	in place + guaranteed	stable, linked lists, external data	simple, fast on small / sorted input

Memory trick: **"Heap = Hierarchy, BST = Bisection"**: a heap only promises parent ≥ children (good for the max); a BST orders left < root < right (good for searching any key).

14 Linear and binary search

Q42: Write the differences between linear search and binary search. Explain the binary search algorithm step by step.

✎ **Asked in:** 17th NTRCA CSE 2023 · BEPZA Programmer 2023 · step by step: Bank AP written (bank name cut off)

Point	**Linear**	**Binary**
Data	any order	**sorted**, random access
Idea	check one by one	halve the range
Best / worst	O(1) / O(n)	O(1) / **O(log n)**
10^6 items	up to 10^6 checks	≤ 20 checks

```
lo = 0; hi = n - 1;
while (lo <= hi) {
  mid = lo + (hi - lo) / 2;
  if (a[mid] == key) return mid;
  if (a[mid] < key) lo = mid + 1;
  else              hi = mid - 1;
}
return -1;
```

→ find 34 in 3 8 15 21 34 42 57: mid 3 (21) → go right · mid 5 (42) → go left · mid 4 (34) found, **3 checks**

Q43: Write the recurrence of binary search and solve it.

✎ **Asked in:** Petrobangla AM(IT) 2022 · complexity (MCQ): Combined Bank Officer(IT) 2018

One comparison, then one half: **$T(n) = T(n/2) + c$**, $T(1) = c$. Unrolling: $T(n) = T(n/2^k) + kc$; $n/2^k = 1$ at $k = \log_2 n \rightarrow T(n) = c \log_2 n + c =$ **$O(\log n)$**. (The printed solution solved merge sort's $2T(n/2) + n$.)

Q44: What is the linear search algorithm? Describe its best and worst case complexity.

✎ **Asked in:** BPSC SAE 2021

Scan a[0], a[1], … until key is found or the array ends. Best: key at a[0], **1 comparison, O(1)**. Worst: last or absent, **n comparisons, O(n)**. Average (n + 1)/2. Works on unsorted data and linked lists.

15 Hashing

Q45: Which data structure gives O(1) expected search time for a set of objects?

✎ **Asked in:** BCC AP 2021 (printed "try yourself") · chaining MCQ: 49th BCS ICT 2025

A **hash table**: h(key) gives the slot directly. Collisions are handled by **chaining** (a list per slot) or open addressing (linear / quadratic probing, double hashing). With load factor $\alpha = n/m$, a search with chaining costs **$O(1 + \alpha)$** on average: O(1) when the table is resized to keep $\alpha \leq 1$. Worst case O(n) if everything collides.

```
h(k) = k mod 7, chaining
0: 14 → 21
1:
3: 10 → 24
5: 5
```

Q46: Compare blind search and intelligent search. What is a heuristic function?

✎ **Asked in:** 45th BCS ICT 2024

Blind (uninformed)	Intelligent (informed, heuristic)
no knowledge of how near the goal is	uses an estimate of the distance to the goal
BFS, DFS, uniform-cost, iterative deepening	greedy best-first, **A***, hill climbing
explores many nodes; simple	far fewer nodes if the heuristic is good

A **heuristic function** h(n) estimates the cost from node n to the goal, e.g. straight-line distance on a map or misplaced tiles in the 8-puzzle. A* uses $f(n) = g(n) + h(n)$ and is optimal when h never overestimates (**admissible**).

16 Divide and conquer vs dynamic programming

Q47: Both divide and conquer and dynamic programming break a problem into smaller instances. What are the differences? How does DP relate to D&C?

✎ **Asked in:** BCC AP 2021 · WZPGCL AE(CSE) 2023 · relation: BPSC AP 2023

Point	Divide and conquer	Dynamic programming
Sub-problems	**independent** (disjoint halves)	**overlapping** (same ones recur)
Repeated work	re-solves if they repeat	solves each once, **stores** it (table / memo)
Direction	top-down recursion	bottom-up table or memoised top-down
Examples	merge sort, quick sort, binary search	Fibonacci, LCS, 0/1 knapsack, Floyd–Warshall

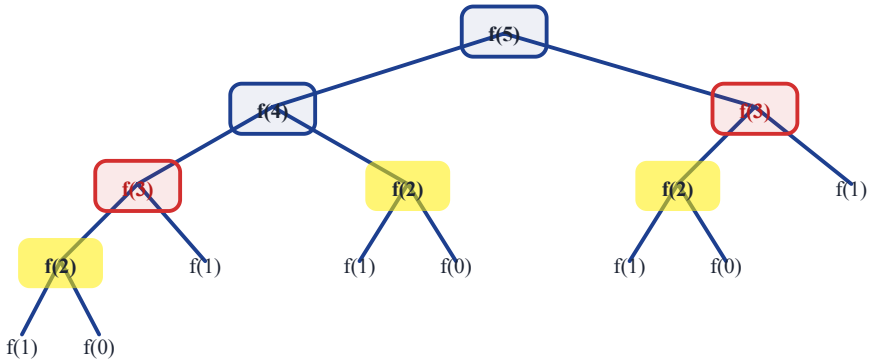
Relation: DP is divide and conquer **plus memory**: when the D&C recursion tree contains the same sub-problem many times, saving its answer turns exponential time into polynomial. MCQ "which use divide and conquer?" ✎ **Asked in:** BPDB AE(CSE) 2021 (options not printed) → merge sort, quick sort, binary search, Strassen.

Q48: Write down the algorithm for computing a Fibonacci number by dynamic programming.
✎ **Asked in:** BPSC ADA 2022 · PSB Programmer 2018

```
long fib(int n) {
    long f[n + 2];
    f[0] = 0; f[1] = 1;
    for (int i = 2; i <= n; i++)
        f[i] = f[i-1] + f[i-2];
    return f[n];
}
```

f: 0 1 1 2 3 5 8 13 21 34 **55** → fib(10) = 55

Plain recursion fib(n − 1) + fib(n − 2) recomputes the same values: **O(2ⁿ)**. The table computes each once: **O(n) time**, O(n) space (O(1) keeping only the last two).



plain recursion: fib(5) makes 15 calls; f(3) is solved twice (red), f(2) three times (yellow). The DP table solves each of f(0) … f(5) once.

Memory trick: DP needs two things: **overlapping sub-problems** (the repeated boxes above) and **optimal substructure**. Top-down with a memo = **memoization**; bottom-up with a table = **tabulation**.

17 Greedy method

Q49: Write the difference between the greedy method and dynamic programming. Does a greedy algorithm always give the optimal solution? If not, when does it?
✎ **Asked in:** 18th NTRCA College ICT 2024 · optimality: BPSC AP 2023

Greedy	Dynamic programming
best-looking choice now, never revisited	tries all sub-choices, keeps the best
fast, little memory	slower, a table
optimal only for some problems	always optimal (if the recurrence is right)
Dijkstra, Kruskal, Huffman, fractional knapsack	0/1 knapsack, LCS, matrix chain

Not always optimal: Coins {1, 3, 4}, amount 6: greedy takes 4 + 1 + 1 (3 coins), optimal is 3 + 3 (2 coins). Greedy is optimal when the problem has the **greedy-choice property** and **optimal substructure** (e.g. fractional knapsack, MST, activity selection).

Q50: What is the time complexity of Huffman coding?

✎ **Asked in:** Combined Bank Officer(IT) 2019

Greedy: put all symbols in a **min-heap** by frequency; repeatedly join the two smallest under a new node (their sum). n − 1 merges × O(log n) heap work → **O(n log n)**. Frequencies a 5, b 9, c 12, d 13, e 16, f 45: merges a+b = 14, c+d = 25, 14+e = 30, 25+30 = 55, f+55 = 100.

Symbol	f	c	d	a	b	e
Code	**0**	100	101	1100	1101	111

→ 224 bits vs 300 with a fixed 3-bit code; no code is a prefix of another.

Q51: n customers request time slots [start, end]; overlapping requests cannot both be served. Write an algorithm to serve the maximum number of customers.

✎ **Asked in:** BPDB AE(CSE) 2023

```
sort requests by END time
last = -infinity; count = 0
for each [s, e] in that order:
    if s >= last:           // no overlap
        serve it; count++; last = e
return count                // O(n log n)
```

[1,4] [3,5] [0,6] [5,7] [3,9] [5,9] [6,10] [8,11] → serve **[1,4] [5,7] [8,11]** = 3 customers

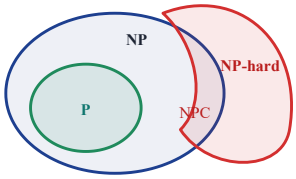
Exam trap: Sorting by **start** time (as a printed solution does) is wrong: [1,10], [2,3], [4,5] → it serves only [1,10], but [2,3] and [4,5] give 2. The earliest **finish** leaves the most room: the classic **activity-selection** greedy.

18 Backtracking and NP

Q52: Define with examples: backtracking, NP, NP-hard and NP-complete problems.

✎ **Asked in:** 45th BCS CSE 2024

Term	Meaning	Example
Backtracking	build a solution step by step, undo a step that breaks a rule	N-queens, sudoku
P	solvable in polynomial time	sorting, shortest path
NP	a solution can be **verified** in polynomial time	SAT, subset sum
NP-hard	every NP problem reduces to it; need not be in NP	TSP optimisation, halting
NP-complete	in NP **and** NP-hard	SAT, 3-colouring, TSP decision



if P ≠ NP

19 Game search and other terms

Q53: What is the minimax ("maximum") algorithm? Define terminal and utility in game playing. What is its limitation?

✎ **Asked in:** 45th BCS ICT 2024 (printed "maximum algorithm")

Minimax searches the game tree: MAX picks the child with the highest value, MIN the lowest, values backed up from the leaves. **Terminal** state = game over (win, loss, draw); **utility** = its numeric payoff (+1, 0, −1). **Limitation**: it explores the whole tree, $O(b^m)$ (chess $b \approx 35$, $m \approx 100$): impossible in practice → cut with **alpha-beta pruning** and a depth limit with an evaluation function.

IDA* ✎ **Asked in:** BUET M.Sc. April 2017: iterative-deepening A*: depth-first with an f = g + h cost bound raised each round. Advantages: optimal like A* with an admissible h, but memory only **O(depth)** instead of A*'s whole frontier.

Parameterized algorithm ✎ **Asked in:** BUET M.Sc.: (complexity theory) runs in f(k) · n^O(1), exponential only in a small parameter k, e.g. vertex cover in $O(2^k n)$. In code, writing an algorithm over parameters (types, comparators) lets one routine serve many inputs: reuse without copies.

20 Infix, prefix, postfix

Q54: Explain infix, prefix and postfix notation, and the method of converting an infix expression.

✎ **Asked in:** BTCL JAM 2022 · BBL AM(ICT) 2019 · expression from the NTRCA question bank

Infix A + B (operator between, needs brackets) · **prefix** (Polish) + A B · **postfix** (reverse Polish) A B +. Pre/postfix need **no brackets** and are evaluated with one stack.

Conversion: scan left to right; operand → output; (→ push;) → pop to (; operator → pop while the top has higher (or equal, left-assoc.) precedence, then push; at the end pop all. ↑ is right-associative.

Infix	Postfix	Prefix
A + B * C	A B C * +	+ A * B C
(A + B) * C	A B + C *	* + A B C
A ↑ B * C − D + E / F / (G + H)	**AB↑C*D−EF/GH+/+**	**+ −*↑ABCD//EF+GH**
(A + B) * C − (D − E) ↑ (F + G)	**AB+C*DE−FG+↑−**	**−*+ABC↑−DE+FG**
A − B / (C * D ↑ E)	**ABCDE↑*/−**	**−A/B*C↑DE**

Stack trace for A − B / (C * D ↑ E) → postfix (the third expression of the NTRCA set):

Token	Stack	Output
A		A
−	−	A
B	−	AB
/	− /	AB
(	− / (	AB
C	− / (	ABC
*	− / (*	ABC
D	− / (*	ABCD
↑	− / (* ↑	ABCD
E	− / (* ↑	ABCDE
)	− /	ABCDE↑*
end		**ABCDE↑*/−**

21 Recursion

Q55: Write a recursive algorithm for the Tower of Hanoi.

✎ **Asked in:** 17th NTRCA ICT 2023

```
hanoi(n, from, via, to):
  if n == 0: return
  hanoi(n-1, from, to, via)
  move disk n: from → to
  hanoi(n-1, via, from, to)
```

Moves $M(n) = 2M(n-1) + 1 = \mathbf{2^n - 1}$. n = 3: A→C A→B C→B A→C B→A B→C A→C (7 moves).

22 Data structures and algorithms

Q56: What is a data structure? Explain linear and non-linear data structures with examples.

✎ **Asked in:** 17th NTRCA ICT 2023 · PSB ADA 2018

A **data structure** is a way of organising data in memory for efficient access and change. **Linear**: elements in a sequence, one next element (array, linked list, stack, queue). **Non-linear**: hierarchy or network, many next elements (tree, graph, heap).

Q57: What is an algorithm? Name three ways to express one. Describe the factors that algorithm complexity depends on.

✎ **Asked in:** BPSC AP 2021 · factors: Combined Bank Officer(IT) 2020

A finite sequence of unambiguous steps that turns input into output and stops. Expressed as **natural language steps, pseudo-code, flowchart** (or a program). Complexity depends on **input size**, input arrangement (best / worst / average), operations counted, data structure, and machine / compiler (constant factors).

MCQ "which runs fastest?" ✎ **Asked in:** 49th BCS ICT 2025: $O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n)$ → **$O(\log n)$**.

Q58: Find the maximum-sum contiguous subarray of A = [−2, 1, −3, −1, 2, 1, −5, 4].

✎ **Asked in:** BPDB AE(CSE) 2023

Kadane: cur = max(x, cur + x), best = max(best, cur); O(n).

→ max sum **4** ([4]); if A[3] = 1 as on one sheet: [1, 2, 1] = 4

A printed answer copies the classic array's [4, −1, 2, 1] = 6, which is not in this A.

Q59: Write int equilibrium(int arr[], int n): an index where the sum of lower-index elements equals the sum of higher-index ones, else −1. {−7, 1, 5, 2, −4, 3, 0} → 3.

✎ **Asked in:** BPDB AE(CSE) 2023

```
int equilibrium(int a[], int n) {
  int total = 0, left = 0;
  for (int i = 0; i < n; i++) total += a[i];
  for (int i = 0; i < n; i++) {
    if (left == total - left - a[i]) return i;
    left += a[i];
  }
  return -1;            /* O(n), not O(n²) */
}
```

→ 3 (−1 = −1) · {1, 2, 3} → −1

Which structure for which job

Job	Structure	Job	Structure
undo, recursion, brackets	stack	scheduling, BFS, buffering	queue
frequent insert / delete	linked list	index by position	array
priority / max quickly	heap	sorted search, range	BST / B+ tree
exact-key lookup O(1)	hash table	networks, roads	graph

★ Night before the exam

1. Height h: max $2^{h+1} - 1$, min h + 1 nodes
2. N levels ≤ $2^N - 1$ nodes; full: leaves = internal + 1
3. Array tree: children 2i, 2i + 1; parent ⌊i/2⌋
4. Pre/post + in → unique; pre + post only if full
5. Inorder of a BST is sorted
6. BST delete, 2 children → inorder successor
7. B+ tree: ≤ n − 1 keys; leaf split copies key up
8. Comparison sort ≥ Ω(n log n): decision tree
9. Quick: n log n avg, n^2 sorted + first pivot
10. Merge: 2T(n/2) + n = n log n, O(n) space
11. Binary search: T(n) = T(n/2) + c = log n
12. BFS → queue (shortest unweighted); DFS → stack
13. Dijkstra: no negative edges; Kruskal E log E
14. Hash chaining O(1 + α)
15. DP = D&C + stored overlapping sub-problems
16. Hanoi $2^n - 1$ moves; Fibonacci DP O(n)

The 49th BCS ICT 2025 DSA MCQs in one box

Question	Answer	Question	Answer
BFS uses	**queue**	traversal that sorts a BST	**inorder**
sorting lower bounds proved by	**decision tree**	not comparison-based	**counting sort**
insertion sort worst comparisons	**n(n − 1)/2**	fastest growth-rate	**O(log n)**
hash chaining, load α	**O(1 + α)**	suffix tree used for	**string search**

Top traps: "construct a BST" from sequences that are not a BST → say so · bubble sort works on unsorted data; binary search doesn't · activity selection sorts by **end** time · a printed BST is not a heap · randomized quicksort is still n^2 in the worst case · DFS is not optimal · binary search on a linked list is O(n)

Average cost	Access	Search	Insert	Delete
Array (unsorted)	1	n	n	n
Linked list	n	n	1 (known node)	1 (known node)
Stack / queue	top / front 1	n	1	1
Hash table	–	1	1	1
BST (balanced / skewed)	–	log n / n	log n / n	log n / n
Binary heap	max 1	n	log n	log n (max)

Memory trick: Complexity ladder "**1 · log · √ · n · n log n · n^2 · 2^n · n!**": read the question's growth, find the rung, answer.

✓ Practice MCQs (asked ones first)

1. Data structure used in BFS: ✎ 49th BCS ICT 2025

(a) stack (b) queue
(c) heap (d) linked list

2. Traversal that gives a BST in sorted order: ✎ 49th BCS ICT 2025

(a) preorder (b) postorder
(c) level order (d) inorder

3. Not a comparison-based sort: ✎ 49th BCS ICT 2025

(a) counting sort (b) merge sort
(c) heap sort (d) quick sort

4. Worst-case comparisons of insertion sort: ✎ 49th BCS ICT 2025

(a) $n(n + 1)$ (b) $n \log n$
(c) $n(n - 1)/2$ (d) $n(n - 1)/4$

5. Hash table, chaining, load factor α: average search ✎ 49th BCS ICT 2025

(a) $O(\alpha^2)$ (b) $O(\log \alpha)$
(c) $O(\alpha)$ (d) $O(1 + \alpha)$

6. Randomized quicksort, worst case: ✎ Combined Bank Officer(IT) 2019

(a) $O(n^2)$ (b) $O(n \log n)$
(c) $O(n)$ (d) $O(\log n)$

7. Max nodes in a binary tree of N levels: ✎ Combined Bank AP 2020

(a) 2^N (b) $2^N - 1$
(c) N^2 (d) $2N$

8. Push 200, 500, 100, then Pop returns: ✎ BAPEX AGM(ICT) 2023

(a) 200 (b) 500
(c) 100 (d) nothing

9. Time complexity of Huffman coding: ✎ Combined Bank Officer(IT) 2019

(a) $O(n)$ (b) $O(n^2)$
(c) $O(\log n)$ (d) $O(n \log n)$

10. Binary search complexity: ✎ Combined Bank Officer(IT) 2018

(a) $O(\log n)$ (b) $O(n)$
(c) $O(n \log n)$ (d) $O(1)$

Answer key → 1-b · 2-d · 3-a · 4-c · 5-d · 6-a · 7-b · 8-c · 9-d · 10-a

Written question: pre[] = {1, 2, 4, 8, 9, 5, 3, 6, 7}, post[] = {8, 9, 4, 5, 2, 6, 7, 3, 1}: draw the binary tree and write its inorder. 10 marks

✎ **Asked in:** BPDB AE(CSE) 2024 · BUET M.Sc. April 2016

1. Root 1 (first of pre, last of post); pre[1] = 2 is the left child 2
2. In post, 8 9 4 5 2 (up to 2) form the left subtree, 6 7 3 the right 3
3. Recurse: 2 → (4 → 8, 9), 5; 3 → 6, 7; draw it (p. 3) 3
4. Inorder **8 4 9 2 5 1 6 3 7**; note pre + post is unique only for a full tree 2

Before the exam ✓

- ✓ rebuild a tree from two traversals
- ✓ complexity table by heart
- ✓ BFS/DFS + Dijkstra trace
- ✓ array vs linked list, stack vs queue